Fundamentos de Comunicação HTTP com Node.js e Express

[1]Departamento de Ciências Exatas e da Terra – Universidade do Estado da Bahia (UNEB) – Salvador – BA – Brazil

Abstract. *This meta-paper describes the style to be used in articles and short papers for SBC conferences. For papers in English, you should add just an abstract while for the papers in Portuguese, we also ask for an abstract in Portuguese ("resumo"). In both cases, abstracts should not have more than 10 lines and must be in the first page of the paper.*

Resumo. *Este trabalho tem como objetivo melhorar, de forma prática, os conceitos fundamentais de redes de computadores por meio do desenvolvimento de uma aplicação baseada no modelo cliente-servidor. Para isso, foi implementado um servidor HTTP utilizando a plataforma Node.js, com suporte ao tratamento de requisições e comunicação com clientes por meio da web. A proposta busca demonstrar, na prática, como ocorre a troca de informações entre cliente e servidor, evidenciando o papel do protocolo HTTP no processo de comunicação.*
Além disso, o projeto permite observar aspectos importantes relacionados ao comportamento das requisições, à estrutura das respostas e à interação entre diferentes componentes de uma aplicação web. Também são considerados elementos que influenciam a adaptação das respostas do servidor, como informações enviadas pelo cliente durante a comunicação.
Dessa forma, o trabalho contribui para a compreensão dos princípios básicos que sustentam o funcionamento da internet, integrando teoria e prática em um contexto aplicado. Os resultados obtidos reforçam a importância do entendimento desses conceitos para o desenvolvimento de aplicações

1. Introdução

All full papers and posters (short papers) submitted to some SBC conference, including any supporting documents, should be written in English or in Portuguese. The format paper should be A4 with single column, 3.5 cm for upper margin, 2.5 cm for bottom margin and 3.0 cm for lateral margins, without headers or footers. The main font must be Times, 12 point nominal size, with 6 points of space before each paragraph. Page numbers must be suppressed.

Full papers must respect the page limits defined by the conference. Conferences that publish just abstracts ask for **one**-page texts.

The first page must display the paper title, the name and address of the authors, the abstract in English and "resumo" in Portuguese ("resumos" are required only for papers written in Portuguese). The title must be centered over the whole page, in 16 point boldface font and with 12 points of space before itself. Author names must be centered in 12 point font, bold, all of them disposed in the same line, separated by commas and with 12 points of space after the title. Addresses must be centered in 12 point font, also with

12 points of space after the authors' names. E-mail addresses should be written using font Courier New, 10 point nominal size, with 6 points of space before and 6 points of space after.

The abstract and "resumo" (if is the case) must be in 12 point Times font, indented 0.8cm on both sides. The word **Abstract** and **Resumo**, should be written in boldface and must precede the text.

2. Tecnologias Utilizadas

In some conferences, the papers are published on CD-ROM while only the abstract is published in the printed Proceedings. In this case, authors are invited to prepare two final versions of the paper. One, complete, to be published on the CD and the other, containing only the first page, with abstract and "resumo" (for papers in Portuguese).

3. Funcionamento do Servidor

Section titles must be in boldface, 13pt, flush left. There should be an extra 12 pt of space before each title. Section numbering is optional. The first paragraph of each section should not be indented, while the first lines of subsequent paragraphs should be indented by 1.27 cm.

3.1. Subsections

The subsection titles must be in boldface, 12pt, flush left.

4. Rotas e Tipos de Requisição

Em aplicações web baseadas em Node.js com o uso do framework Express, as rotas são responsáveis por definir como o servidor responde a diferentes requisições feitas pelo cliente. Cada rota está associada a um endereço (URL) e a um método HTTP específico, determinando a ação que será executada quando uma requisição é recebida.

Os principais métodos HTTP utilizados neste projeto são o GET e o POST. O método GET é utilizado para solicitar dados ao servidor, sem causar alterações no estado da aplicação. Ele é comumente empregado para recuperar informações, como páginas HTML ou dados em formato JSON. Já o método POST é utilizado para enviar dados ao servidor, geralmente com o objetivo de criar ou modificar recursos, sendo amplamente utilizado em formulários e envio de informações.

No Express, a definição de rotas é realizada de forma simples e intuitiva, por meio de funções que associam um caminho a um método HTTP. Por exemplo, uma rota definida como `app.get('/')` indica que o servidor responderá a requisições GET feitas para a raiz da aplicação. Da mesma forma, uma rota como `app.post('/dados')` permite o recebimento de dados enviados pelo cliente.

Além disso, as rotas podem ser organizadas de forma estruturada, seguindo padrões como APIs RESTful, onde cada endpoint representa um recurso da aplicação. Nesse modelo, diferentes métodos HTTP são utilizados sobre a mesma rota para representar operações distintas, como leitura, criação, atualização e remoção de dados.

Outro aspecto importante é o tratamento das requisições recebidas. O Express fornece objetos como `req` (requisição) e `res` (resposta), que permitem acessar informações

enviadas pelo cliente, como parâmetros, corpo da requisição e cabeçalhos, além de possibilitar o envio de respostas apropriadas ao cliente.

Dessa forma, o uso adequado de rotas e métodos HTTP é fundamental para a organização e funcionamento da aplicação, permitindo uma comunicação clara, eficiente e estruturada entre cliente e servidor.

Tabela 1. Comportamento das requisições na aplicação

URL	Método HTTP	Comportamento
Qualquer rota	GET	Registra informações da requisição e retorna JSON
Qualquer rota	POST	Captura o corpo da requisição e retorna JSON
Qualquer rota	Outros	Processa e retorna resposta padrão em JSON

Diferentemente de aplicações estruturadas com múltiplas rotas específicas, o servidor desenvolvido neste projeto utiliza um único manipulador de requisições, sendo capaz de processar qualquer URL acessada pelo cliente. Independentemente do método HTTP utilizado, o servidor registra informações relevantes da requisição, como método, URL e cabeçalhos, além de capturar o corpo da mensagem quando presente.

A Tabela 1 resume o comportamento geral da aplicação para diferentes tipos de requisição.

5. Engenharia Reversa

A engenharia reversa é o processo de análise de um sistema com o objetivo de compreender sua estrutura, funcionamento e comportamento interno a partir de sua observação externa, geralmente sem acesso ao código-fonte original. Essa prática é amplamente utilizada em áreas como segurança da informação, interoperabilidade de sistemas e análise de protocolos de comunicação. Segundo [Eilam 2005], a engenharia reversa permite a extração de conhecimento a partir de artefatos existentes, sendo uma ferramenta essencial para a compreensão de sistemas complexos.

No contexto de aplicações web, a engenharia reversa é frequentemente aplicada na análise da comunicação entre cliente e servidor, que ocorre por meio do protocolo HTTP. De acordo com [Fielding 2000], a interação entre sistemas distribuídos na web é baseada em requisições e respostas, o que possibilita a observação detalhada do comportamento de uma aplicação por meio do tráfego de rede.

Neste projeto, a engenharia reversa pode ser realizada utilizando ferramentas como o painel de desenvolvedor dos navegadores (DevTools), que permitem monitorar e inspecionar todas as requisições HTTP realizadas pelo cliente. Por meio dessas ferramentas, é possível analisar elementos como métodos de requisição (GET, POST), URLs acessadas, parâmetros enviados, códigos de status retornados e cabeçalhos HTTP.

A análise dos cabeçalhos HTTP desempenha um papel fundamental nesse processo. Informações como o campo *User-Agent* permitem identificar características do cliente, enquanto outros cabeçalhos, como *Content-Type* e *Authorization*, revelam detalhes sobre o formato dos dados e possíveis mecanismos de autenticação utilizados pela aplicação. Segundo [Fielding, R. and Reschke, J. 2014], os cabeçalhos HTTP são essenciais para a correta interpretação das mensagens trocadas entre cliente e servidor.

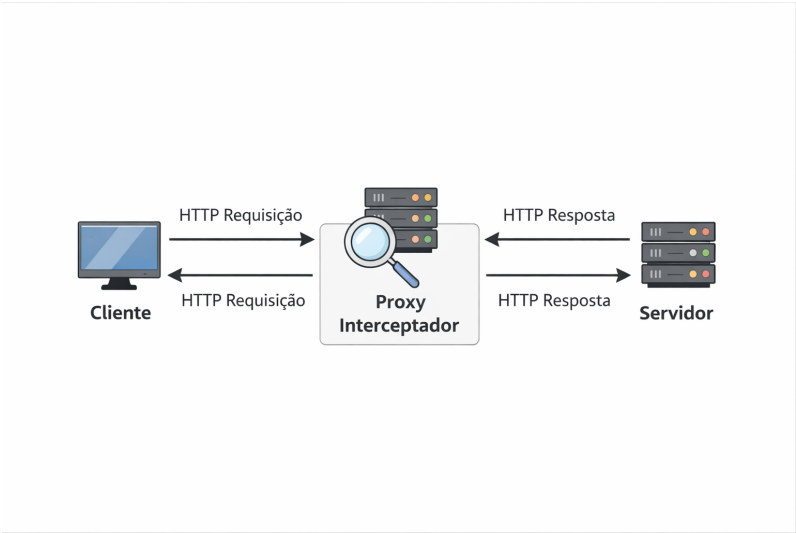

Figura 1. Fluxo de interceptação de requisições HTTP utilizando um proxy intermediário, permitindo análise e modificação das mensagens entre cliente e servidor.

Além disso, a engenharia reversa possibilita a identificação das rotas disponíveis em uma aplicação web. Ao observar as requisições realizadas, é possível mapear endpoints, inferindo os tipos de operações suportadas pelo servidor. Esse processo, conhecido como *API mapping*, é amplamente utilizado para documentar sistemas que não possuem especificação formal ou documentação disponível.

Outro aspecto importante é a análise do formato das respostas retornadas pelo servidor, geralmente estruturadas em JSON. A partir dessas respostas, pode-se compreender a organização dos dados, suas relações e possíveis regras de negócio implementadas no backend. Essa abordagem permite reconstruir parcialmente a lógica da aplicação, contribuindo para testes, integração com outros sistemas e auditorias.

Entretanto, é importante destacar que a engenharia reversa deve ser conduzida de maneira ética e legal. Conforme discutido em [Chikofsky and Cross 1990], o uso dessa técnica deve respeitar restrições legais, direitos autorais e políticas de uso dos sistemas analisados, sendo recomendada principalmente em contextos acadêmicos, educacionais ou com autorização explícita.

Portanto, a aplicação da engenharia reversa neste projeto permite uma compreensão mais aprofundada da comunicação baseada em HTTP, das rotas implementadas no servidor Node.js e do comportamento geral da aplicação. Essa análise contribui significativamente para o aprendizado prático dos conceitos de redes de computadores e desenvolvimento web, além de reforçar a importância da observação e interpretação de sistemas reais.

6. Trabalhos Futuros

Como continuidade deste projeto, diversas melhorias podem ser implementadas com o objetivo de tornar a aplicação mais robusta, escalável e alinhada às práticas modernas de desenvolvimento web.

Uma das principais evoluções consiste na implementação de um sistema de

autenticação e autorização, utilizando mecanismos como tokens JWT (*JSON Web Tokens*). Essa abordagem permite controlar o acesso a recursos da aplicação de forma segura, garantindo que apenas usuários autenticados possam realizar determinadas operações.

Outra melhoria relevante é a integração com um banco de dados, como o MongoDB ou o MySQL, possibilitando a persistência de dados. Isso permitiria transformar a aplicação em um sistema mais completo, com armazenamento de informações e suporte a operações CRUD (*Create, Read, Update, Delete*).

Além disso, a evolução para uma arquitetura baseada em APIs RESTful pode tornar a aplicação mais modular e interoperável. Segundo [Fielding 2000], esse modelo arquitetural promove escalabilidade e separação de responsabilidades entre cliente e servidor, sendo amplamente adotado em sistemas distribuídos modernos.

Também é possível considerar a utilização de containers, como o Docker, para facilitar o processo de deploy e garantir consistência entre diferentes ambientes de execução. Essa abordagem contribui para a portabilidade e simplifica a distribuição da aplicação.

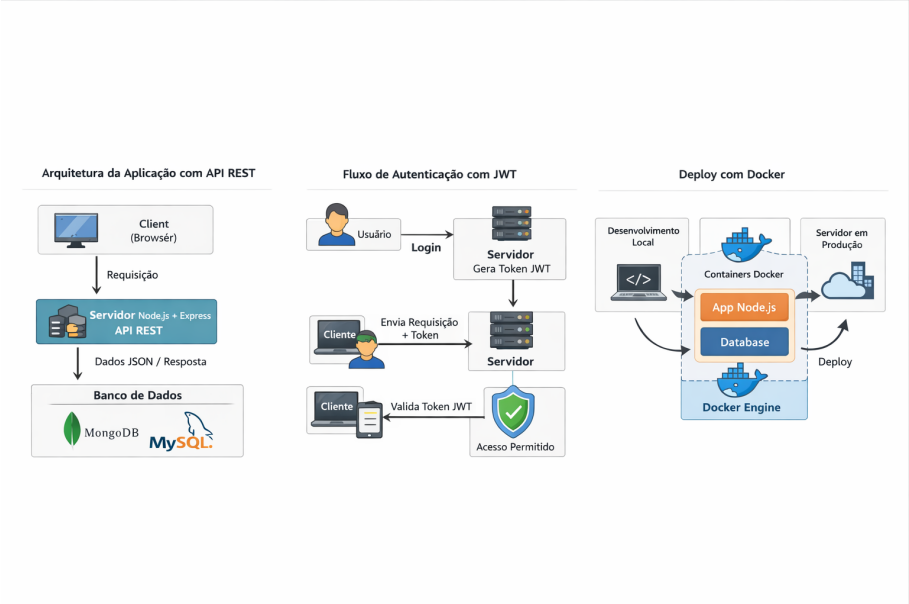

Figura 2. Ilustração de possíveis futuras aplicações.

Por fim, melhorias relacionadas ao desempenho podem ser exploradas, como o uso de balanceamento de carga e otimizações no tratamento de requisições HTTP. A adoção de práticas como cache de respostas e compressão de dados pode reduzir significativamente o tempo de resposta do servidor, melhorando a experiência do usuário.

Referências

Chikofsky, E. J. and Cross, J. H. (1990). Reverse engineering and design recovery: A taxonomy. *IEEE Software*, 7(1):13–17.

Eilam, E. (2005). *Reverse Engineering*. Wiley Publishing.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures.

Fielding, R. and Reschke, J. (2014). Hypertext transfer protocol (http/1.1): Message syntax and routing. RFC 7230.